8.7 Evict 流程

AMBA CHI · 模块 8 · 请求流程

中文全文译本 | 原文：VLSI Mentor，8.7. Evict Flow | 2026-11-13

阅读说明：本文按用户提供的 12 页 PDF 翻译，保留原文的 18 个小节、图示、三种 HDL 示例、断言和 Testbench。协议操作码、信号名及代码逻辑保留英文；代码注释译为中文。原文第 3 节只有标题，未补写内容。原文包含若干教学简化及协议错误，正文以【译注 N】标记，末尾集中对照 Arm CHI Issue H 说明。原文的示例代码与“预期输出”不表示已通过编译或验证。

Evict 流程概述

WriteBack 将脏缓存行送回内存；Evict 则是它的轻量版本：丢弃一条不需要写回的干净缓存行，因为内存中已经保存了相应的值。整个过程不传输数据。【译注 1】

既然不传数据，为什么还要通知 Home 节点？因为 Home 维护一份目录，记录哪些缓存持有哪些缓存行。如果缓存悄悄丢弃一条缓存行，目录就仍会记录一个已经不存在的持有者。Evict 帮助保持目录准确，使 Home 不必向已经不存在的副本发出无用的 snoop，也有助于避免侦听过滤器触发不必要的反向失效。【译注 2】

这个流程很简单：一个请求、一个完成响应，没有数据传输，缓存行进入 Invalid 状态。Evict 只用于干净缓存行；脏缓存行必须写回。本文展示的是代表性的教学模型，并非协议规范。【译注 3、4】

难度：中级 · 预计阅读时间：14 分钟

关键词：AMBA CHI、Evict、目录、侦听过滤器、干净缓存行。

章节串联：8.6 讲的是脏缓存行的写回；本章讲的是干净缓存行的丢弃，不需要传输数据。两者构成这一教学模型中的两种淘汰流程。8.8 将介绍 MakeUnique：不重新获取数据，将 Shared 权限升级为 Unique。【译注 6】

1. 学习目标

学完本章后，你应该能够：

- 说明 Evict 会丢弃一条本地持有的干净缓存行（UC/SC），该缓存行不需要写回。
- 解释 Evict 为什么不传输数据：按原文的简化模型，内存中已经保存了相应的值。【译注 1】
- 描述 Evict 的真正用途：提高 Home 节点目录的准确性。
- 区分 Evict 与 WriteBack：干净与脏、不传数据与传数据。
- 分析静默丢弃干净缓存行为什么会使目录保留陈旧信息。
- 使用 SystemVerilog、Verilog-2001 和 VHDL 实现一个代表性的 Evict 决策模型。

2. 为什么要学习这个内容?

大多数缓存淘汰针对的是干净缓存行：数据被读入后从未被修改，内存中仍保存着同样的内容。丢弃这样的缓存行不会丢失数据，也不需要写回。不过，让 Home 知道这次淘汰仍然有好处，而这就是 Evict 的作用：用一个请求和一个完成响应完成低成本通知，不传输数据。【译注 1】

这里要理解的是，一致性硬件不仅跟踪数据，也跟踪“谁持有什么”。Home 的目录，以及经常为它提供支持的侦听过滤器，会决定每次事务需要侦听哪些缓存。如果记录与实际情况逐渐偏离，Home 就会把 snoop 浪费在已经不再持有缓存行的缓存上；在精确过滤器的实现中，也可能为了给陈旧条目腾出位置，反向失效真正仍在使用的缓存行。

Evict 有助于保持记录准确。理解它，就是理解怎样通过目录记账，让 snoop 有明确目标、让过滤器避免被陈旧条目占用。这是一项性能机制；如果忽视它，可能影响整个系统的性能。【译注 2、3】

3. 关键术语

【原文此处只有标题，没有术语条目。】

4. 与上一章的联系

8.6 章的 WriteBack 淘汰一条脏缓存行。按照原文描述，在完整的 DBID 握手流程中，缓存必须先把数据送到内存，才能放弃该缓存行。Evict 是这一对淘汰流程的另一半：干净缓存行的情况。【译注 5】

由于干净缓存行的数据已经在内存中，Evict 不携带数据，也不需要 DBID；它只有一个 REQ 和一个 Comp。它与 WriteBack 的共同点是最终缓存状态都为 Invalid，并且都要求选择适当的流程：脏行使用 WriteBack，干净行使用 Evict。【译注 1、3】

这里还引入了此前读写流程没有特别强调的一点：这个流程的目的并非移动数据，而是让 Home 的目录保持准确。

5. 核心概念：丢弃干净缓存行，通知 Home，不传数据

Evict 丢弃请求方持有的一条干净缓存行，并通知 Home，以便维持目录的准确性。它不传输数据，最终缓存状态为 Invalid。

- 仅用于干净行。缓存行为 UC 或 SC，即干净且与内存一致，因此无需保存其数据；如果是脏行，就应使用第 8.6 章的 WriteBack。【译注 1、3】
- 发出请求。缓存向 Home 发送 Evict。没有 DBID，也没有数据，因为没有内容需要写回。
- 更新目录。Home 从该缓存行的目录持有者集合及侦听过滤器中移除请求方，后续就不会再为这条缓存行侦听该缓存。【译注 2】
- 完成。Home 返回 Comp。缓存将该行转为 I，并释放缓存条目。【译注 4：发送 Evict 前就必须已经进入 I。】

综合起来，原文想表达的是：Evict 通过一个不带数据的 REQ → Comp 交换，通知 Home 本地已丢弃一条干净缓存行。其目的在于目录准确性：Home 从持有者集合中移除请求方，使后续 snoop 可以跳过该缓存。WriteBack 移动脏数据；Evict 只传递状态信息。脏行使用 WriteBack，干净行使用 Evict。【译注 1、2、3】

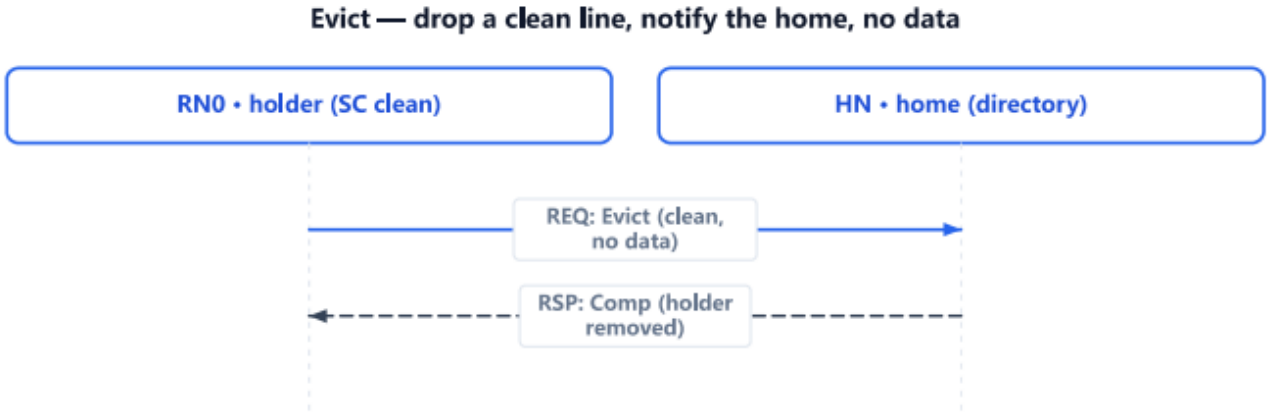
6. 工程类比：丢弃未做批注的副本，通知登记台

再次以办公室文件为例：这一次你只是阅读文件，你的副本没有做任何修改，对应一条干净缓存行。

- 你不必提交任何修改，因为主副本已经包含你这份副本中的全部内容。你只需要回收自己的副本。
- 但你会告诉前台：“47 号文件我已经用完了。”这就相当于 Evict 通知。前台的借阅登记不再把你列为持有者。
- 如果你不通知前台就把副本粉碎，登记表仍认为你持有它。之后有人需要从所有持有者处收回文件时，前台会无谓地联系你。如果登记表已满，前台甚至可能为了腾出记录槽位，收回你正在使用的另一份文件，因为它误以为你还持有其实已经丢弃的那份。

副本本身不值得继续保留，因为它是干净副本；但借阅登记很重要。Evict 就像一张礼貌的通知单，帮助登记保持准确。

7. 工程图示：一次 Evict



图中文字：RN0 · holder (SC clean) = RN0，持有 SC 干净副本；HN · home (directory) = HN，Home 节点及其目录；REQ: Evict (clean, no data) = REQ 通道上的 Evict，不携带数据；RSP: Comp (holder removed) = RSP 通道上的 Comp，原图将其解释为持有者已移除。【译注 2、4】

图 1：一条干净缓存行的 Evict。RN0 持有处于 SC 状态的干净缓存行，丢弃它并在 REQ 通道发送 Evict。原文认为内存已保存该值，因此不需要数据。Home 从其目录中该行的持有者集合移除 RN0，然后返回 Comp；RN0 转入 Invalid。整个流程只有一个请求和一个完成响应，没有数据，也没有 snoop。【译注 1、2、4】

两个消息阶段，不传数据：缓存宣布要丢弃一条干净缓存行，Home 更新目录并确认，缓存行进入 Invalid。对比原文所说包含 DBID 分配和数据传输的四阶段 WriteBack，Evict 相当于去掉了数据部分，因为没有脏数据需要保存。

8. Evict 与 WriteBack 的比较

以下并列比较原文教学模型中的两种淘汰流程：

比较项	Evict（8.7）	WriteBack（8.6）
缓存行状态	干净：UC / SC	脏：UD / SD
传输的数据	无	脏缓存行的数据
握手流程	REQ → Comp	REQ → DBID → CopyBackWrData
用途	提高目录准确性	保存脏数据
缓存最终状态	I	I

【表格译注】此表保留原文简化写法。CopyBackWrData 是原文使用的缩写，本译本对照的 IHI0050H 使用 CopyBackWriteData；表中的 DBID 阶段对应的具体响应需依照规范，例如 CompDBIDResp，不能将表格视为完整事务状态机。Evict 在发送时的协议初始状态为 I；“UC/SC”描述的是本地决定淘汰前的状态。【译注 4】

原文希望读者记住的规则是：用脏状态决定选择哪个流程。脏行需要 WriteBack，否则数据会丢失；干净行使用 Evict，因为数据已在内存中，无需传输，流程只是为了维持目录准确性。两者结束时缓存状态都为 I，区别在于是否传输数据。【译注 1、3】

9. 为什么还要通知？为了目录准确性

Evict 不传输数据，因此它的价值就在通知本身。

- Home 跟踪持有者。目录通常由侦听过滤器支持，记录每条缓存行被哪些缓存持有，使 ReadUnique 或写事务只向实际持有者发出 snoop。
- 静默丢弃使记录偏离。如果缓存不发 Evict 就丢弃干净缓存行，目录仍将它列为持有者，形成“幽灵持有者”，即副本已不存在但目录仍保留的记录。Home 随后可能侦听一个已不持有该行的缓存。
- 无用 snoop 浪费带宽。每次不必要的 snoop 都消耗互连资源，事务还需要等待“本地不存在该行”的响应。
- 过滤器可能反向失效。精确侦听过滤器的容量有限，幽灵条目会占用槽位。为了接纳新缓存行，过滤器可能反向失效真正仍在使用的缓存行，从而淘汰缓存当前需要的数据。静默丢弃因此可能增加抖动。【译注 2、3】

这里的核心观点是：Evict 虽然不带数据，其传递的信息却有助于提高目录准确性。一致性硬件依据“谁持有什么”的记录来发送 snoop，并管理有限的过滤器容量。如果记录高估了持有者数量，把已经离开的持有者仍列在其中，Home 就可能承担多余 snoop 的开销，甚至反向失效仍在使用的缓存行。

Evict 是减轻这类偏差的一种低成本手段：一个请求、一个完成响应，帮助目录接近实际状态。缓存行副本已经不值得保留，但“它已经离开”的信息仍然有价值。【译注 2、3】

10. 流程演示：淘汰一条干净的 SC 缓存行

RN0 持有一条 SC 状态的干净缓存行，需要把它占用的条目用于新缓存行。

1. 选择流程。缓存行是干净的，因此原文选择 Evict；脏缓存行则需要 WriteBack。不会传输数据。
2. REQ。RN0 向 HN 发送 Evict。没有 DBID，也没有数据拍。
3. 更新目录。HN 从该缓存行的持有者集合及侦听过滤器中移除 RN0，之后就不再为此行侦听 RN0。【译注 2】
4. Comp。HN 返回 Comp。原文描述 RN0 此时执行 SC → I，然后复用缓存条目。【译注 4：真实协议要求在第 2 步发送前完成到 I 的转换。】

最终状态：RN0 不再持有该缓存行；在原文模型中，目录准确反映了这一点，整个过程没有传输数据。另一个核之后发起 ReadUnique 时，不会再因此行侦听 RN0。如果跳过第 2 步，静默丢弃副本，目录就会保留一个幽灵 RN0，这就是后面 DebugLab 讨论的场景。【译注 2、3】

11. RTL / 硬件视角：Evict 决策

请求方的 Evict 决策逻辑很小：淘汰干净行时通知 Home；淘汰脏行时转交 WriteBack；Evict 本身永不发送数据。以下是代表性的串行教学模型。

【代码译注】以下三份代码保留原文逻辑，只翻译注释并整理换行。特别是 invalidated 在 Comp 后才有效的设计，与真实缓存行必须在发送 Evict 前进入 I 的要求不符。不能直接将它们用于 RN DUT 或合规 VIP；完整说明见译注 7。

SystemVerilog

原文代码署名：Azvya Education Pvt. Ltd. / VLSI Mentor。

```
// 请求方 Evict 决策的代表性教学示例。
// 淘汰 CLEAN 行时发出 Evict：通知 Home，不传数据。
// DIRTY 行必须转交 WriteBack，否则会丢失数据。
// 原文模型在 Comp 后才置 Invalid；该时机见译注 4、7。
module chi_evict_txn (
  input  logic clk, rst_n,
  input  logic evict,          // 本缓存希望丢弃该行
  input  logic is_dirty,       // Dirty -> WriteBack，不发 Evict
  input  logic comp,           // Home 确认 Evict
  output logic evict_req,      // 发出 Evict，通知 Home
  output logic route_writeback,// 脏行改走 WriteBack
  output logic sends_data,     // Evict 永不携带数据
  output logic invalidated     // 原文：Comp 后丢弃到 I
);
  typedef enum logic { IDLE, WAIT_COMP } state_t;
  state_t state, next;

  always_comb begin
    next = state;
    case (state)
      IDLE:      if (evict && !is_dirty) next = WAIT_COMP;
      WAIT_COMP: if (comp)               next = IDLE;
    endcase
  end

  // 干净行淘汰时通知；脏行交给 WriteBack，不直接丢弃。
  assign evict_req       = (state == IDLE) && evict && !is_dirty;
  assign route_writeback = (state == IDLE) && evict &&  is_dirty;
  assign sends_data      = 1'b0;
  assign invalidated     = (state == WAIT_COMP) && comp;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else        state <= next;
endmodule
```

Verilog-2001

同样的行为，用 Verilog-2001 表示如下：

```
// 请求方 Evict 决策的代表性示例（Verilog-2001）。
module chi_evict_txn (
  input clk, rst_n, evict, is_dirty, comp,
  output evict_req, route_writeback, sends_data, invalidated
);
  localparam IDLE = 1'b0, WAIT_COMP = 1'b1;
  reg state, next;

  always @* begin
    next = state;
    case (state)
      IDLE:      next = (evict && !is_dirty) ? WAIT_COMP : IDLE;
      WAIT_COMP: next = comp ? IDLE : WAIT_COMP;
    endcase
  end

  assign evict_req       = (state == IDLE) && evict && !is_dirty;
  assign route_writeback = (state == IDLE) && evict &&  is_dirty;
  assign sends_data      = 1'b0;
  assign invalidated     = (state == WAIT_COMP) && comp;

  always @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else        state <= next;
endmodule
```

VHDL

```
-- 请求方 Evict 决策的代表性示例（VHDL）。
library ieee;
use ieee.std_logic_1164.all;
entity chi_evict_txn is
  port (
    clk, rst_n      : in  std_logic;
    evict, is_dirty : in  std_logic;
    comp            : in  std_logic;
    evict_req       : out std_logic;
    route_writeback: out std_logic;
    sends_data      : out std_logic;
    invalidated     : out std_logic
  );
end entity;

architecture rtl of chi_evict_txn is
  type state_t is (IDLE, WAIT_COMP);
  signal state : state_t := IDLE;
begin
  process (clk, rst_n)
  begin
    if rst_n = '0' then
      state <= IDLE;
    elsif rising_edge(clk) then
      case state is
        when IDLE =>
          if evict = '1' and is_dirty = '0' then
            state <= WAIT_COMP;
          end if;
        when WAIT_COMP =>
          if comp = '1' then
            state <= IDLE;
          end if;
      end case;
    end if;
  end process;

  evict_req <= '1' when
    (state = IDLE and evict = '1' and is_dirty = '0') else '0';
  route_writeback <= '1' when
    (state = IDLE and evict = '1' and is_dirty = '1') else '0';
  sends_data <= '0';
  invalidated <= '1' when
    (state = WAIT_COMP and comp = '1') else '0';
end architecture;
```

三个版本都在淘汰干净行时通过 evict_req 发出通知；脏行改走 WriteBack；Evict 路径从不发送数据。下面的 DebugLab 将演示干净行淘汰省略通知时可能发生的情况。

12. 验证视角：仅干净行、发通知、不传数据

原文通过以下属性检查其 Evict 教学模型：只针对干净行发出、始终通知 Home、从不传输数据。【译注 3、7】

```
// 绑定到 chi_evict_txn。
// 1. 只为 CLEAN 行发出 Evict；脏行转交 WriteBack。
property p_evict_only_clean;
  @(posedge clk) disable iff (!rst_n)
    evict_req |-> !is_dirty;
endproperty

// 2. 干净行淘汰总是发出通知，不静默丢弃。
property p_notify_on_clean_evict;
  @(posedge clk) disable iff (!rst_n)
    (evict && !is_dirty && !route_writeback) |-> evict_req;
endproperty

// 3. Evict 不带数据；脏行淘汰转交 WriteBack。
// 下式按原文保留，存在 SVA 组合语法问题，见译注 7。
property p_no_data_and_dirty_routed;
  @(posedge clk) disable iff (!rst_n)
    (sends_data == 1'b0) && ((evict && is_dirty) |-> route_writeback);
endproperty
```

这些检查之外，原文强调的系统层面要点是：Evict 不携带数据，因此正确性取决于所传达的信息。必须准确表达两个事实：这条行是干净的，不欠任何数据；它现在已经不在本地，因此目录可以删除该持有者。如果第一点错了，用 Evict 丢弃脏行，就会丢失本应由 WriteBack 保存的数据；如果第二点没通知，目录可能留下一个幽灵持有者。

原文将这两种情况概括为 Home 对系统状态的模型未保持准确，而非单纯的数据路径问题。Evict 的意义在于：即使不移动数据，也可以维护状态记录。【译注 3：合法的静默淘汰并非通用协议错误。】

- 这些属性想证明什么：Evict 只用于干净行、始终发出通知、不传输数据。
- 它们不能证明什么：Home 确实更新了目录；那属于 Home 一侧的行为。
- 原文给出的故障特征：干净行被淘汰却未发送 Evict，目录中留下幽灵持有者。【译注 2、3、7】

13. Testbench：干净行通知，脏行转交 WriteBack

测试先淘汰一条干净缓存行，检查发出了通知且没有数据传输；再确认脏缓存行会转交 WriteBack。

【译注 7】以下 Testbench 保留原文驱动与检查逻辑，存在仿真调度竞争风险。“预期输出”来自原文，不是本译本运行得到的结果。

```
module tb_chi_evict_txn;
  logic clk = 0, rst_n = 0, evict = 0, is_dirty = 0, comp = 0;
  logic evict_req, route_writeback, sends_data, invalidated;
  int errors = 0;

  chi_evict_txn dut (.*);
  always #5 clk = ~clk;
  initial begin
    @(posedge clk) rst_n = 1;

    // 干净行：发出 Evict、不传数据、不转交 WriteBack。
    @(posedge clk) begin evict = 1; is_dirty = 0; end
    if (!evict_req || route_writeback || sends_data) begin
      errors++;
      $display("FAIL clean evict: req=%0b wb=%0b data=%0b",
               evict_req, route_writeback, sends_data);
    end
    else $display("PASS clean line: Evict issued, no data");
    @(posedge clk) evict = 0;

    // 原文模型：完成后才置为 Invalid。
    @(posedge clk) comp = 1;
    if (!invalidated) begin
      errors++;
      $display("FAIL not invalidated on Comp");
    end
    else $display("PASS invalidated after Comp");
    @(posedge clk) comp = 0;

    // 脏行：不发 Evict，改走 WriteBack。
    @(posedge clk) begin evict = 1; is_dirty = 1; end
    if (evict_req || !route_writeback) begin
      errors++;
      $display("FAIL dirty line: req=%0b wb=%0b",
               evict_req, route_writeback);
    end
    else $display("PASS dirty line: routed to WriteBack, no Evict");
    @(posedge clk) evict = 0;

    if (errors == 0) $display("ALL TESTS PASSED");
    else             $display("%0d FAILURE(S)", errors);
    $finish;
  end
endmodule
```

原文预期输出：

```
PASS clean line: Evict issued, no data
PASS invalidated after Comp
PASS dirty line: routed to WriteBack, no Evict
ALL TESTS PASSED
```

中文含义：干净行测试通过，已发 Evict 且无数据；Comp 后失效检查通过；脏行已转交 WriteBack，未发 Evict；所有测试通过。

14. DebugLab：静默丢弃干净缓存行

案例 1：干净行未发送 Evict 就被丢弃，形成幽灵持有者和无用 snoop。

现象

snoop 流量和反向失效次数远高于预期。各个核正在使用的热缓存行被迫淘汰。数据始终正确，但性能随着时间推移下降，并且缓存压力越大，问题越明显。

证据

目录仍列出实际上已经不再持有该缓存行的节点：

```
RN0 持有多条 SC 缓存行，随后在本地丢弃它们，未发送 Evict。
目录仍将 RN0 列为这些行的持有者，形成幽灵记录。
之后：RN2 发起 ReadUnique(L)
  -> HN 向 RN0 发 snoop，因为目录仍记录 RN0。
  -> RN0 返回“本地不存在”。
结果：每个事务可能产生多余 snoop。

过滤器被幽灵条目填满。
  -> HN 反向失效 RN0 中一条仍在使用的热缓存行，以回收槽位。
  -> RN0 被迫重新获取一条本来不希望失去的缓存行。
```

每次静默丢弃都可能留下一个 Home 仍会据此采取行动的陈旧条目。

最早出现偏离的位置

缓存把淘汰当作纯本地行为，在没有发送 Evict 的情况下丢弃干净缓存行。从那一刻起，Home 的目录就高估了持有者集合；每次静默丢弃都可能生成一个没有及时删除的幽灵条目。

根因

Evict 的用途是帮助目录保持准确，而非传输数据。干净缓存行不需要写回，但 Home 仍在跟踪谁持有它。如果没有 Evict 通知，Home 无法通过这次淘汰获知缓存已丢弃副本。

结果是目录高估：幽灵持有者引发不必要的 snoop，在某些精确过滤器实现中还会造成对真实缓存行的反向失效。它与第 8.6 章 WriteBack 中的数据保留故障不同：后者会丢失数据；这里数据安全，但 Home 对持有者的记录偏离实际。【译注 2、3】

修复方法

原文建议每次淘汰干净行都发出 Evict，使 Home 可以从持有者集合中删除请求方。示例中干净行淘汰时的 evict_req 就承担这个通知作用。目录准确时，Home 可以只侦听实际持有者，过滤器尽量保留仍有效的条目，减少无用 snoop 和不必要的反向失效。【译注 2、3：这是该教学案例的优化策略，不是普遍的协议强制要求。】

15. 常见错误

- 静默丢弃干净行。错误假设：不传数据就不需要通知。原文所讨论的问题是产生幽灵持有者；其建议是在干净行淘汰时发送 Evict。【译注 3】
- 用 Evict 丢弃脏行。错误假设：任何缓存行都可以直接丢弃。问题是脏数据丢失；脏行的普通淘汰应使用 WriteBack。【译注 3】
- Evict 时发送数据。错误假设：淘汰就一定要传输缓存行。问题是引入不必要的传输；Evict 不携带数据。
- 等待 DBID。错误假设：Evict 属于写数据事务。结果是等待一个不会到来的 DBID 分配；Evict 的基本流程是 REQ → Comp，不需要 DBID。
- 没有进入 Invalid。错误假设：缓存还能保留该副本。结果是本地仍存在旧的持有状态；Evict 要求请求方进入 I。【译注 4：而且必须在发送前进入。】
- 以为 Evict 能保存数据。错误假设：Evict 是一种简化 WriteBack。结果是丢失脏数据；Evict 不负责保存脏数据。

16. 工程检查清单

以下按原文列出；其中的教学假设见译注。

- 只对干净缓存行（UC/SC）使用 Evict；脏行使用 WriteBack。【译注 3】
- 不发送数据，也不等待 DBID；Evict 是 REQ → Comp。
- 干净行淘汰时发出 Evict，不要静默丢弃。【译注 3：不是通用 CHI 强制要求。】
- 让 Home 从其目录或侦听过滤器中移除持有者。【译注 2：HN 可以忽略该 hint。】
- Comp 后将缓存行转到 I。【译注 4：原文时机错误；应在发送 Evict 前进入 I。】
- 提高目录准确性，减少无用 snoop 和不必要的反向失效。

17. 核心要点

- 原文将 Evict 描述为丢弃一条本地持有的干净缓存行（UC/SC），不需要写回，因为内存中已保存该值。【译注 1】
- Evict 不传数据；基本流程是 REQ → Comp，最终缓存状态为 I。【译注 4】
- 其用途是提高目录准确性，让 Home 能够从持有者集合中移除请求方。【译注 2】
- 静默丢弃干净行可能留下幽灵持有者，引发无用 snoop；某些精确过滤器实现还可能产生额外反向失效。【译注 3】
- 原文通过“是否为脏”选择普通淘汰流程：脏行 WriteBack，干净行 Evict。【译注 3】
- 干净行淘汰时通知、不传数据、帮助目录记录保持准确；本文展示的是代表性模型。

18. 快速复习

Evict 流程：丢弃一条本地持有的干净缓存行（原文以 UC/SC 为例）。按原文模型，内存已经保存相同的值，因此无需写回。整个流程不传数据，只进行 REQ Evict → Comp，缓存最终为 Invalid。【译注 1、4】

它是 WriteBack 的轻量对应流程。原文用脏状态来选择：脏行必须通过 WriteBack 保存数据，干净行使用 Evict。Evict 的目标不是移动数据，而是提高目录准确性：Home 从该缓存行的持有者集合和侦听过滤器中移除请求方，从而避免再次为此行侦听该缓存。【译注 2、3】

原文要避免的情形是：未发送 Evict 就静默丢弃干净行。目录保留幽灵持有者，引发不必要的 snoop；精确侦听过滤器被陈旧条目占满时，可能反向失效仍在使用的缓存行，造成抖动。数据虽然仍安全，但 Home 的记录与实际情况不再一致。原文将其概括为“仅干净行、不传数据、始终通知”，并再次强调这是代表性的模型。【译注 3】

下一章是 8.8 MakeUnique 流程：不重新取回数据，将 Shared 升级为 Unique。【译注 6】

接下来：第 8.8 章 MakeUnique 流程

Evict 和 WriteBack 结束了缓存行在本地的驻留；MakeUnique 则在原地改变它的权限。原文预告第 8.8 章将介绍：当处理器已经持有 Shared 缓存行并希望写入时，如何使其他共享副本失效，从而获得 Unique 所有权，而不重新获取已经持有的数据；为什么这能节省一次完整数据传输；以及它与重新获取缓存行的 ReadUnique 有何不同。原文将其称为从读取者变为写入者的高效路径。【译注 6：MakeUnique 有保证覆盖整条缓存行的使用限制。】

标准与规范

依据标准：Arm AMBA CHI Architecture Specification。

该规范定义 CHI 节点类型、一致性协议，以及节点之间的事务流程。Mesh 拓扑、Home 节点的布局和性能调优属于实现选择。

原文还讨论了 RTL 结构、验证方法和调试技术。这些是基于标准形成的工程实践，并非标准本身施加的要求。

译注：与 CHI 规范的区别

本节是译者补充，不属于原文。依据为本知识库保存的 Arm IHI0050H《AMBA CHI Architecture Specification》。原文用于理解“无数据淘汰通知”的思路较直观，但不能直接用作 RN 或 HN VIP 的合规依据。

译注 1：Clean 不等于 DDR 中一定已有同一个值

Clean 的关键是该缓存不承担更新主存的责任。SC 可以和其他缓存中的 SD 共存，此时 SC 副本也可能比主存更新。

例：RN0 持有 SC，值为 7；RN1 持有 SD，值为 7；DDR 的值仍为 3。RN0 可以丢弃自己的 SC 副本，因为 Dirty 责任由 RN1 承担；不是因为 DDR 已经等于 7。

依据：IHI0050H，B1.5 和 B4.1；本地 CHI.md 约 L1495、L8001–L8004。

译注 2：Evict 是 hint，Comp 不保证目录已经删除持有者

规范明确：HN 可以不更新 Snoop Filter 或 Snoop Directory 就返回 Comp。因此，原文“Comp 表示持有者已经移除”“之后一定不会再 snoop 此 RN”等说法都过于绝对。Evict 给 HN 提供更新目录的机会，不是必须执行该更新的命令。

依据：IHI0050H，B5.2.4 / 图 B5.14，规范印刷页 299；本地 CHI.md 约 L12400–L12417。

译注 3：Clean 行允许静默淘汰，不能一概判为协议错误

规范 B4.6 允许某些状态变化不通知系统其他部分；表 B4.36 包含 UC、UCE、SC 到 I 的缓存淘汰。发起事务可以让这一行为在接口上可见，但并非每次都强制发送。

因此，原文“每个 Clean 淘汰都必须 Evict”“不发 Evict 就是 bug”的说法，只适用于它选择的教学模型或某个具体实现的性能策略。目录保留一个可能已经不存在的持有者会导致额外 snoop，但不能仅凭这一点认定数据一致性出错。

原文的“脏行 WriteBack、干净行 Evict”也是普通淘汰场景的简化分类。干净数据还可能通过 WriteEvictFull 或 WriteEvictOrEvict 送往下一级；特殊的显式 Cache Invalidate 行为也不能与普通替换淘汰混为一谈。若启用 MTE，还必须遵守标签的 Dirty 保留规则。

依据：IHI0050H，B4.6 / 表 B4.36；本地 CHI.md 约 L10383–L10409。

译注 4：发送 Evict 前就必须进入 I

规范流程为：本地缓存行先转入 I → RN 发送 Evict → HN 返回 Comp_I → 双方释放相应请求资源。

所以协议表格中 Evict 的初始状态为 I。原文第 5、7、10、11、13、16 节把“收到 Comp”作为缓存行才开始失效的时机，这是错误的。可以等 Comp 后退休 Evict 事务，但不能让真实缓存行保持有效直到 Comp 才失效。

Evict 不携带数据，不要求 DBID；ExpCompAck 必须为 0，不发送 CompAck。

依据：IHI0050H，B4.7.2、B5.2.4，以及表 B4.8；本地 CHI.md 约 L10949–L10954、L12413–L12417、L8555。

译注 5：WriteBack 不等于必须等待数据真正写入 DDR

写回数据可以由下一级缓存或下游节点接收，Dirty 数据的保存责任可以转移到那里。不能把“RN 完成写回事务”解释为“DDR 必须已经更新”。具体完成条件需要按对应事务及握手规则判断。

依据：IHI0050H，CopyBack 写请求与 WriteBack 流程说明；本地 CHI.md 约 L8755–L8761、L12513–L12517。

译注 6：MakeUnique 不是任意 Shared 行的普通写权限升级

Requester 只有在保证对整个缓存行的所有字节进行写入时，才能使用 MakeUnique。这一限制使其可以不依赖旧数据。因此，原文下章预告不能直接理解为“已有 Shared 数据、想修改其中一个字节时，总可以发送 MakeUnique”。

依据：IHI0050H，B4.2 的 MakeUnique 请求说明；本地 CHI.md 约 L8377–L8380。

译注 7：示例 RTL、SVA 与 Testbench 的适用边界

- 三份 RTL 都把 invalidated 定义为 WAIT_COMP 且收到 comp。如将该信号解释为真实缓存行刚刚失效，就与译注 4 的规范要求冲突。它至多可以重新解释成“Evict 请求退休指示”，但那已不再是原文给它的含义。
- 第 12 节只有 property 声明，没有展示 assert property 的实例。仅有 property 声明并不会自动执行断言检查。
- p_no_data_and_dirty_routed 把布尔 && 与包含 |-> 的 property 表达式直接混用，存在 SVA 语法问题。实际实现宜分别为“无数据”和“脏行转交 WriteBack”建立合法的 property/assert。
- p_notify_on_clean_evict 没有限定模型处于 IDLE 或已接受输入。若 busy 时 evict 仍可为 1，它可能要求 RTL 此时也输出 evict_req，与原代码行为不符；需要明确输入握手约束。
- Testbench 在 posedge 使用阻塞赋值驱动输入，随即读取组合输出，还在同一类采样沿释放复位，存在调度竞争及组合逻辑尚未更新的风险。实际验证应采用清晰的驱动/采样时序，例如 clocking block 或错开的驱动边沿。
- 代码未包含真实 CHI 链路信用、请求字段、Retry、事务 ID、同地址 snoop 冲突等逻辑；它只是教学决策模型。此译本没有将代码宣称为已编译或已验证通过。

来源与对照

翻译来源：用户提供的 evict.pdf，12 页；页面标题为“8.7. Evict Flow”，页脚记录网页 https://www.vlsimentor.com/chi/evict-flow。PDF 中的打印时间为 2026/11/13 15:18。

规范对照：Arm IHI0050H，AMBA CHI Architecture Specification，Issue H。对应本知识库 CHI/00_spec/CHI.md 及同目录原始规范 PDF。上面的行号用于本地定位，正式引用以章节、表格和图号为准。

原文图 1 保留；表格已翻译排版。原文署名、章节关系与预期输出均保留。本文新增的规范核查内容全部标为译注。